

СОДЕРЖАНИЕ

	Введение	2
	Основная часть	3
1	Модели машинного обучения и их сериализация	3
1.1	Постановка задачи и описание данных	3
1.2	Предобработка данных и состав моделей	3
1.3	Сериализация моделей	4
2	Разработка веб-дашборда	4
2.1	Структура приложения	4
2.2	Фрагменты исходного кода	5
2.3	Размещение проекта	5
3	Демонстрация результатов прогнозирования	5
3.1	Прогнозирование на типичных данных	5
3.2	Прогнозирование на данных с выбросами	6
	Заключение	7
	Список использованных источников	8
	Приложение А	9
	Приложение Б	11

ВВЕДЕНИЕ

Актуальность темы связана с необходимостью применения обученных моделей машинного обучения к новым данным и представления результатов прогнозирования в форме, доступной для анализа [4, 5]. В рамках дисциплины «Машинное обучение и большие данные» рассматривается полный цикл разработки ML-приложения: подготовка данных, обучение моделей, сериализация и инференс через веб-интерфейс.

Объектом исследования является набор данных `data/csgo_done.csv`, содержащий снимки состояния раундов CS:GO. Целевая переменная `bomb_planted` определяет факт установки бомбы (0/1). Решается задача **бинарной классификации** при дисбалансе классов: доля положительного класса составляет около 11%.

Для реализации использованы библиотеки Python: `scikit-learn`, `CatBoost`, `pandas`, `NumPy`, `Plotly` и `Streamlit` [1, 2, 3].

Цель работы: разработать веб-дашборд для инференса моделей классификации и анализа данных CS:GO.

Задачи:

- 1) выполнить предобработку данных, обучить и сериализовать модели, оценить качество по F1, ROC-AUC и PR-AUC;
- 2) реализовать дашборд на `Streamlit` с четырьмя разделами: сведения о разработчике, описание данных, визуализации, инференс;
- 3) подготовить репозиторий, примеры CSV для демонстрации прогнозов и оформить отчёт.

ОСНОВНАЯ ЧАСТЬ

1 Модели машинного обучения и их сериализация

1.1 Постановка задачи и описание данных

Каждая запись набора данных описывает состояние раунда в фиксированный момент времени. Объём выборки $N = 118\,088$ наблюдений. Используются признаки `time_left`, `ct_score`, `t_score`, `map`, а также агрегированные показатели команд СТ и Т: здоровье, броня, деньги, число шлемов, количество живых игроков, наличие наборов для разминирования.

Для оценки качества моделей применяются метрики ROC-AUC, PR-AUC (Average Precision) и F1-score при пороге 0,5. Разбиение на обучающую и тестовую выборки выполняется в соотношении 80/20 с параметрами `stratify=bomb_planted` и `random_state=42`.

1.2 Предобработка данных и состав моделей

Числовые признаки нормализуются с помощью `StandardScaler`. Категориальный признак `map` кодируется `OneHotEncoder(handle_unknown=ignore)`. Конвейер `ColumnTransformer` объединён с классификатором в `Pipeline`, что обеспечивает идентичность предобработки при обучении и инференсе.

Обучение выполняется скриптом `jobs/fit_all_models.py`. Результаты эксперимента фиксируются в файле `storage/classifiers/registry.txt`. Обучены пять моделей:

- **ML1 (logreg)** --- логистическая регрессия (`class_weight=balanced`);
- **ML2 (gbm)** --- градиентный бустинг (`GradientBoostingClassifier`);
- **ML3 (catboost)** --- `CatBoostClassifier`;
- **ML4 (random_forest)** --- случайный лес (`RandomForestClassifier`);
- **ML5 (adaboost)** --- `AdaBoostClassifier` на деревьях решений.

Таблица 1 – Качество моделей на тестовой выборке

Модель	Алгоритм	ROC-AUC	PR-AUC	F1@0.5
ML1	logreg	0.974	0.768	0.740
ML2	gbm	0.997	0.968	0.925
ML3	catboost	0.997	0.970	0.930
ML4	random_forest	0.997	0.965	0.928
ML5	adaboost	0.996	0.964	0.918

Наилучшие значения PR-AUC и F1 показывает модель ML3 (CatBoost).

1.3 Сериализация моделей

Модели scikit-learn и CatBoost сохраняются модулем `pickle` (HIGHEST_PROTOCOL) в каталог `storage/classifiers/` с расширением `.pkl`. Каждый файл содержит полный Pipeline, включая этап предобработки.

Файл `registry.txt` содержит параметры эксперимента (`target`, `rows`, `positive\_rate`) и сведения о каждой модели: идентификатор, наименование, имя файла, значения метрик AUC, AP и F1 на тестовой выборке.

2 Разработка веб-дашборда

2.1 Структура приложения

Дашборд реализован на Streamlit. Точка входа --- `app/main.py`. Страницы расположены в каталоге `app/pages/`:

- `1\_about.py` --- сведения о разработчике;
- `2\_dataset.py` --- описание набора данных и метрики моделей;
- `3\_visualizations.py` --- визуализация данных (Plotly);
- `4\_inference.py` --- инференс выбранной модели.

Модули `backend/csgo\_dataset.py` и `backend/classifi`

er_tools.py содержат функции загрузки данных, построения конвейера предобработки, вычисления метрик и работы с реестром моделей.

2.2 Фрагменты исходного кода

Фрагмент расчёта прогноза на странице инференса:

```
proba = proba_from_estimator(model, raw)
out["proba_bomb_planted"] = proba
out["pred_bomb_planted"] = (proba ≥ 0.5).astype(int)
```

Фрагмент сохранения модели после обучения:

```
file_name = f"{model_id}.pkl"
save_pickle(pipe, STORAGE_DIR / file_name)
save_registry({
    "target": TARGET,
    "rows": int(len(df)),
    "positive_rate": round(float(y.mean()), 4),
    "models": registry_models,
})
```

На странице инференса реализованы: выбор модели, загрузка файла CSV, ручной ввод признаков, вывод вероятности $P(\text{bomb_planted} = 1)$ и класса при пороге 0.5.

2.3 Размещение проекта

Исходный код размещён в репозитории: https://github.com/MOR7E/ML_RGR.

Веб-приложение опубликовано на Streamlit Cloud: <https://morseew.streamlit.app/>.

3 Демонстрация результатов прогнозирования

3.1 Прогнозирование на типичных данных

Файл report/data/csgo_examples_typical.csv содержит наблюдения с типичными значениями признаков. Данные используются для

проверки работы интерфейса загрузки CSV и формирования столбцов proba_bomb_planted и pred_bomb_planted.

Прогнозирование на корректных (типичных) данных (ML3)

time_left	ct_score	t_score	map	ct_health	t_health	ct_money	t_money	ct_players_alive	t_players_alive	pred_proba_bomb_planted	pred_bomb_planted
114.91	8	8	de_inferno	500	500	5250	350	5	5	0.0	0
114.93	6	6	de_train	500	500	300	6600	5	5	0.0	0
114.95	5	6	de_overpass	500	500	6600	5150	5	5	0.0	0
114.92	6	7	de_train	500	500	1450	7550	5	5	0.0	0
94.92	6	7	de_train	500	465	850	7550	5	5	0.0	0

Рисунок 1 – Инференс на типичных данных (csgo_examples_typical.csv)

3.2 Прогнозирование на данных с выбросами

Файл report/data/csgo_examples_outliers.csv содержит наблюдения с экстремальными значениями признаков. Данные используются для оценки поведения моделей в нетипичных игровых ситуациях.

Прогнозирование на данных с выбросами (ML3)

time_left	ct_score	t_score	map	ct_health	t_health	ct_money	t_money	ct_players_alive	t_players_alive	pred_proba_bomb_planted	pred_bomb_planted
7.61	17	18	de_nuke	21	0	1400	0	1	0	0.999	1
14.6	17	17	de_vertigo	317	0	24200	0	4	0	0.9794	1
4.43	17	18	de_mirage	200	0	1100	0	2	0	0.9967	1
12.88	17	17	de_nuke	130	0	4550	0	2	0	0.9974	1
11.92	18	20	de_dust2	117	0	1850	0	2	0	0.9984	1

Рисунок 2 – Инференс на данных с выбросами (csgo_examples_outliers.csv)

ЗАКЛЮЧЕНИЕ

В работе выполнена бинарная классификация события `bomb_planted` по данным CS:GO. Обучены и сериализованы пять моделей машинного обучения, проведена оценка качества на тестовой выборке. Реализован веб-дашборд на `Streamlit` с разделами описания данных, визуализации и инференса. Наилучший результат по метрикам PR-AUC и F1 показала модель ML3 (CatBoost).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Pedregosa F. et al. Scikit-learn: Machine Learning in Python // *Journal of Machine Learning Research*. - 2011. - Vol. 12. - P. 2825--2830. - URL: <http://scikit-learn.org/stable/> (дата обращения: 28.05.2026).
- [2] Streamlit documentation. - URL: <https://docs.streamlit.io/> (дата обращения: 28.05.2026).
- [3] Prokhorenkova L. et al. CatBoost: unbiased boosting with categorical features // *Advances in Neural Information Processing Systems*. - 2018. - Vol. 31. - URL: <https://catboost.ai/> (дата обращения: 28.05.2026).
- [4] Machine Learning Inference - All You Need to Know. - URL: <https://pareto.ai/blog/machine-learning-inference> (дата обращения: 28.05.2026).
- [5] Machine Learning Inference. - URL: <https://www.gigaspace.com/data-terms/machine-learning-inference> (дата обращения: 28.05.2026).

ПРИЛОЖЕНИЕ А

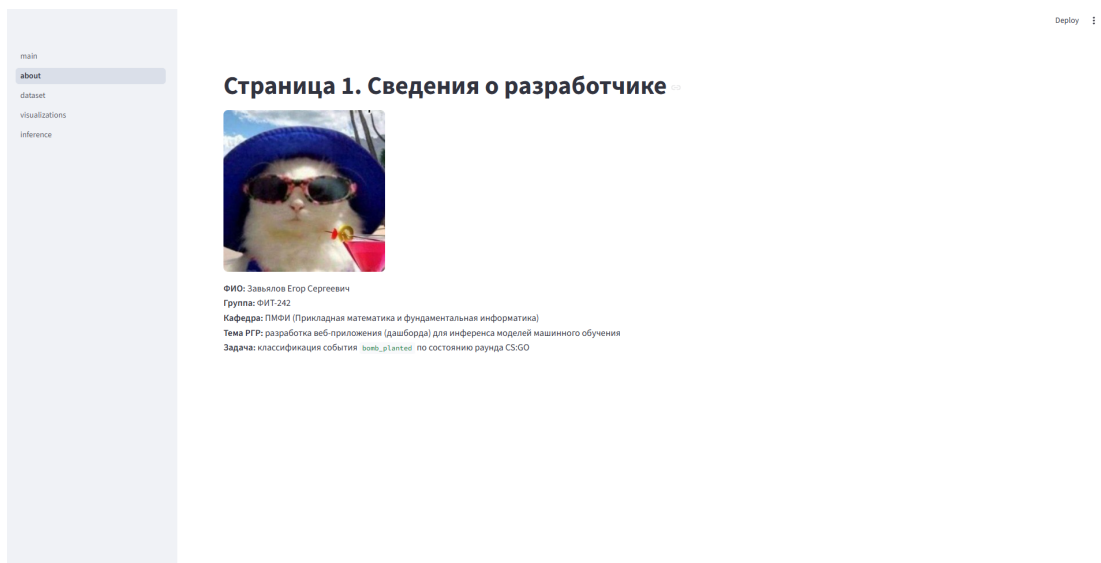


Рисунок 3 – Главная страница дашборда



Рисунок 4 – Страница описания набора данных

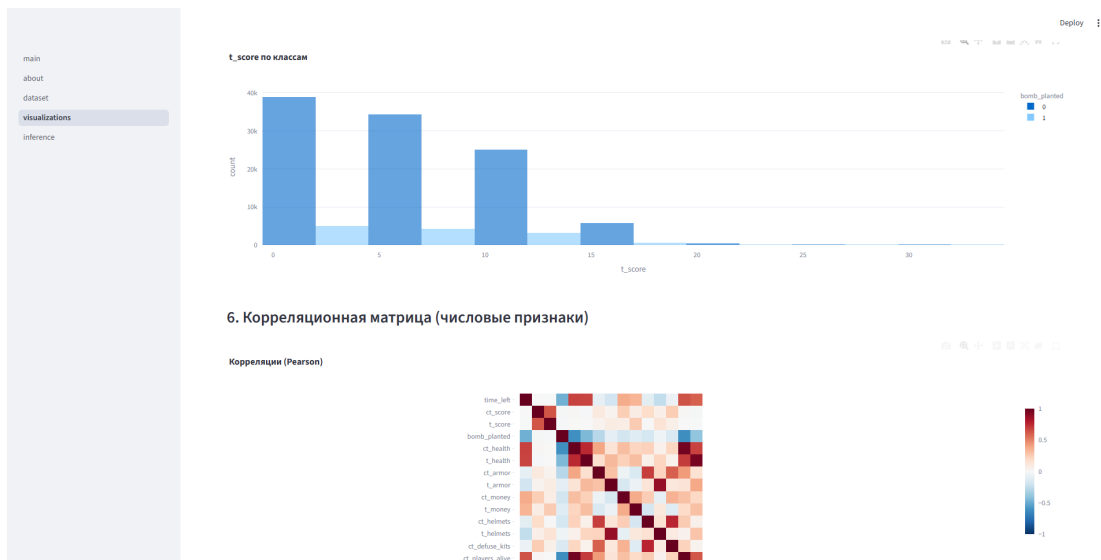


Рисунок 5 – Страница визуализаций

The figure shows a web application interface for inference. The left sidebar has a menu with 'main', 'about', 'dataset', 'visualizations', and 'inference' (selected). The main content area is titled 'Страница 4. Инференс'. It shows a model selection dropdown set to 'ML2 — Gradient Boosting (sklearn)'. Below this, there are two tabs: 'Загрузка CSV' and 'Ручной ввод (1 строка)'. The 'Ручной ввод' tab is active, showing a form with input fields for 14 features: time_left, ct_health, ct_score, t_health, t_score, ct_armor, map, t_armor, de_dust2, ct_money, ct_players_alive, t_money, ct_defuse_kits, ct_helmets, and t_helmets. Each input field has a value and a range indicator.

Рисунок 6 – Страница инференса

ПРИЛОЖЕНИЕ Б

app/pages/1_about.py

```
import streamlit as st
from pathlib import Path

st.title("Страница 1. Сведения о разработчике")

PHOTO_PATH = Path(__file__).resolve().parents[2] / "assets" / "dev.png"
if PHOTO_PATH.exists():
    st.image(str(PHOTO_PATH), width=280)

st.markdown(
    """ФИО
    **:** Завьялов Егор СергеевичГруппа
    **:** ФИТ-242Тема
    ** РГР:** вебдашборд- для инференса моделей MLЗадача
    **:** классификация `bomb_planted` (CS:GO)
    """
)
```

jobs/fit_all_models.py (фрагмент)

```
for model_id, (title, pipe) in catalog.items():
    pipe.fit(split.X_train, split.y_train)
    metrics = compute_metrics(split.y_test, proba_from_estimator(pipe, split.X_test)
    )
    save_pickle(pipe, STORAGE_DIR / f"{model_id}.pkl")
    registry_models[model_id] = {
        "name": title,
        "file": f"{model_id}.pkl",
        "metrics": metrics.as_dict(),
    }
save_registry({"target": TARGET, "rows": len(df), "models": registry_models})
```